

TP : Validation des Données

Ce TP est conçu pour vous guider pas à pas dans l'implémentation des fonctionnalités de validation des données d'entrée dans Laravel 12, qui couvre les bases de la validation, les requêtes de formulaires, les validateurs manuels, les messages d'erreur, les règles disponibles, la validation de fichiers et de mots de passe, ainsi que les règles personnalisées.

Nous allons créer une application simple de blog où les utilisateurs peuvent créer des articles (posts). Chaque post aura un titre (title) et un corps (body). Nous implémenterons la validation pour s'assurer que :

- Le titre est requis, unique dans la base de données, et ne dépasse pas 255 caractères.
- Le corps est requis.

Par la suite, nous étendrons cela avec des fonctionnalités avancées comme les requêtes de formulaires, la validation de fichiers (e.g., une image pour le post), de mots de passe (pour une simulation d'inscription utilisateur), et une règle personnalisée (e.g., titre en majuscules).

1 : Création du Projet Laravel

1. Installez un nouveau projet Laravel 12 via Composer :

```
composer create-project laravel/laravel 012-blog-validation --prefer-dist
```

2. Accédez au dossier du projet :

```
cd 012-blog-validation
```

3. Configurez la base de données dans le fichier .env :

```
DB_CONNECTION=sqlite
```

Pour MySQL, ajustez DB_HOST, DB_PORT, DB_DATABASE, DB_USERNAME, DB_PASSWORD.

4. Lancez le serveur de développement :

```
php artisan serve
```

Accédez à <http://127.0.0.1:8000> pour vérifier que tout fonctionne.

2 : Création du Modèle, Migration et Routes de Base

1. Créez un modèle Post avec une migration et un contrôleur :

```
php artisan make:model Post -m -c
```

Cela génère :

- app/Models/Post.php (modèle).
- database/migrations/xxxx_create_posts_table.php (migration).

- app/Http/Controllers/PostController.php (contrôleur).
- 2. Éditez la migration database/migrations/xxxx_create_posts_table.php pour ajouter les champs :

```
public function up(): void
{
    Schema::create('posts', function (Blueprint $table) {
        $table->id();
        $table->string('title')->unique();
        $table->text('body');
        $table->timestamps();
    });
}
```

- 3. Exécutez la migration :

```
php artisan migrate
```

- 4. Définissez les routes dans routes/web.php :

```
use App\Http\Controllers\PostController;

Route::get('/posts/create', [PostController::class, 'create']);
Route::post('/posts', [PostController::class, 'store']);
```

3 : Création de la Vue pour le Formulaire

- 1. Créez un fichier resources/views/posts/create.blade.php :

```
<!DOCTYPE html>
<html>
<head>
    <title>Créer un Post</title>
</head>
<body>
    <h1>Créer un Post</h1>

    @if ($errors->any())
        <div style="color: red;">
            <ul>
                @foreach ($errors->all() as $error)
                    <li>{{ $error }}</li>
                @endforeach
            </ul>
        </div>
    @endif

    <form method="POST" action="/posts">
        @csrf
        <label for="title">Titre :</label>
        <input type="text" name="title" value="{{ old('title') }}"><br>
        @error('title')
            <div style="color: red;">{{ $message }}</div>
        @enderror

        <label for="body">Corps :</label>
        <textarea name="body">{{ old('body') }}</textarea><br>
        @error('body')
            <div style="color: red;">{{ $message }}</div>
        @enderror

        <button type="submit">Envoyer</button>
    </form>
```

```
</body>
</html>
```

Cela inclut la repopulation des champs avec `old()` et l'affichage des erreurs.

4 : Implémentation de la Validation Basique dans le Contrôleur

1. Éditez `app/Http/Controllers/PostController.php` :

```
namespace App\Http\Controllers;

use Illuminate\Http\Request;
use App\Models\Post;

class PostController extends Controller
{
    public function create()
    {
        return view('posts.create');
    }

    public function store(Request $request)
    {
        $validated = $request->validate([
            'title' => 'required|unique:posts|max:255',
            'body' => 'required',
        ]);

        Post::create($validated);

        return redirect('/posts/create')->with('success', 'Post créé !');
    }
}
```

- Si la validation échoue, Laravel redirige automatiquement avec les erreurs.
 - Utilisez `bail` pour arrêter à la première erreur : `'title' => 'bail|required|unique:posts|max:255'`.
2. Testez : Accédez à `/posts/create`, soumettez un formulaire vide. Vous devriez voir des erreurs affichées.

5 : Utilisation des Requêtes de Formulaires (Form Requests)

Pour des validations plus complexes:

1. Créez une requête de formulaire :

```
php artisan make:request StorePostRequest
```

2. Éditez `app/Http/Requests/StorePostRequest.php` :

```
namespace App\Http\Requests;

use Illuminate\Foundation\Http\FormRequest;

class StorePostRequest extends FormRequest
{
    public function authorize(): bool
    {
        return true; // Autorisez pour ce TP (ajoutez de la logique si
nécessaire, e.g., auth).
    }
}
```

```

public function rules(): array
{
    return [
        'title' => 'required|unique:posts|max:255',
        'body' => 'required',
    ];
}

public function messages(): array
{
    return [
        'title.required' => 'Le titre est requis.',
        'body.required' => 'Le corps est requis.',
    ];
}
}

```

3. Mettez à jour le contrôleur pour utiliser cette requête :

```

public function store(StorePostRequest $request)
{
    $validated = $request->validated();
    Post::create($validated);
    return redirect('/posts/create')->with('success', 'Post créé !');
}

```

- Cela encapsule la validation.

6 : Validateurs Manuels et Données Validées

Pour une validation manuelle :

1. Dans le contrôleur, au lieu de \$request->validate, utilisez :

```

use Illuminate\Support\Facades\Validator;

$validator = Validator::make($request->all(), [
    'title' => 'required|unique:posts|max:255',
    'body' => 'required',
]);

if ($validator->fails()) {
    return redirect('/posts/create')->withErrors($validator)->withInput();
}

$validated = $validator->validated(); // Ou $request->safe()->all();
Post::create($validated);

```

2. Pour les sacs d'erreurs nommés : Utilisez withErrors(\$validator, 'post') et accédez via \$errors->post->first('title').

7 : Validation de Fichiers et Mots de Passe

1. Ajoutez un champ fichier au formulaire : Ajoutez à create.blade.php :

```

<label for="image">Image :</label>
<input type="file" name="image">

```

2. Mettez à jour les règles dans StorePostRequest.php :

```
use Illuminate\Validation\Rule;
use Illuminate\Validation\Rules\File;

'image' => [
    'nullable',
    File::image()->max(1024)->dimensions(Rule::dimensions()->maxWidth(1000)->maxHeight(500)),
],
```

- Stockez le fichier après validation : `$request->file('image')->store('public/images');`.
- 3. Pour les mots de passe : Créez un contrôleur UserController et ajoutez :

```
use Illuminate\Validation\Rules>Password;

$request->validate([
    'password' => ['required', 'confirmed', Password::min(8)->letters()->mixedCase()->numbers()->symbols()],
]);
```

8 : Règles de Validation Personnalisées

1. Créez une règle personnalisée :

```
php artisan make:rule Uppercase --invokable
```

2. Éditez app/Rules/Uppercase.php :

```
namespace App\Rules;

use Closure;
use Illuminate\Contracts\Validation\InvokableRule;

class Uppercase implements InvokableRule
{
    public function __invoke(string $attribute, mixed $value, Closure $fail): void
    {
        if (strtoupper($value) !== $value) {
            $fail('Le :attribute doit être en majuscules.');
        }
    }
}
```

3. Utilisez-la dans les règles : `'title' => ['required', new \App\Rules\Uppercase()]`.

9 : Tests et Améliorations

- Testez avec des données invalides (e.g., titre vide, duplicata).
- Consultez la liste des règles disponibles sur <https://laravel.com/docs/12.x/validation#available-validation-rules>.
- Pour les champs optionnels, utilisez nullable.